

Unicode y codificación UTF-8

Arquitectura de Computadoras, UTN Facultad Regional Villa María

28 de agosto de 2023

Índice

1. Consideraciones iniciales	1
1.1. Codificación y decodificación	1
1.2. Representación de datos	1
1.3. Breve introducción comparativa con ASCII	1
2. Unicode	2
3. Formatos de representación	2
3.1. Codificación mediante UTF-8	2
3.2. Codificar los identificadores Unicode	3

1. Consideraciones iniciales

A los fines de poder comprender el resto del artículo en el contexto de la representación de datos se ofrecen las siguientes consideraciones iniciales.

1.1. Codificación y decodificación

Para el presente artículo se considera codificar al proceso de tomar carácter y asignar un valor binario que lo represente, mientras que decodificar es el proceso de restaurar el carácter original.

1.2. Representación de datos

Cuando hablamos de representación de datos es la manera en que la computadora almacena y transmite los datos, fundamentalmente por dos motivos:

1. La conversión puede resultar en un dato periódico o irracional
2. Es deseable establecer un formato para simplificar su manejo, que normalmente involucra si es de tamaño fijo o variable, y para el caso de representaciones variables la forma de ordenar los bytes y como se debe identificar la longitud.

1.3. Breve introducción comparativa con ASCII

ASCII fue creado en el año 1961 para representar fundamentalmente los símbolos utilizados en el idioma inglés y caracteres de control. El formato es fijo requiriendo 8 bits, el primer bit es 0 cuando se utiliza el estándar original y es 1 cuando se hace uso de ASCII extendido que se utiliza para símbolos de otros idiomas como puede ser los de origen latín, medio oriente o asiáticos entre otros. El principal problema de ASCII, al ser de tamaño fijo, es la limitada cantidad de símbolos que es posible representar, $2^8 = 256$ símbolos, siendo completamente insuficiente teniendo en cuenta la variedad de idiomas y símbolos de diversos orígenes que existen en la actualidad.

Unicode por contraparte no limita la cantidad de símbolos y son añadidos nuevos con cierta frecuencia, además de ofrecer formatos y codificaciones variables no limitando el tamaño del conjunto de símbolos que es posible representar.

Algo importante a mencionar es que Unicode respeta las representaciones de los primeros 128 símbolos definidos en ASCII estándar, siendo en consecuencia equivalentes y compatibles en ambas codificaciones, es decir que cualquiera de los 128 símbolos de la tabla ASCII son codificados de la misma manera en Unicode.

2. Unicode

Unicode es un estándar de codificación que asigna un identificador o código a un símbolo de forma única sin importar idioma u origen.

Esto brinda grandes ventajas, dado que la codificación ya no depende del idioma del dispositivo, ya que el código es único y no hay ambigüedades en su representación, ya no siendo necesario especificar el idioma a decodificar.

A los fines de resumir y simplificar es sencillamente un número único expresado en formato hexadecimal por ejemplo el emoji del mate es representado por el identificador único $(1F9C9)_{16}$ y usualmente se escribe de manera U+1F9C9 a los fines de denotar que es un identificador Unicode.

3. Formatos de representación

Unicode además define el formato en el que serán codificados estos identificadores, es decir como serán representados los identificadores, uno de los más utilizados y el que vamos a utilizar en el contexto de la asignatura es UTF-8, dado a su amplia adopción y posibilidad de comprensión en base a los conceptos ya vistos.

3.1. Codificación mediante UTF-8

UTF-8 es un formato de codificación variable donde se agrupan bloques de 8 bits dando el nombre a esta especificación. Dado que es un formato variable debe existir una manera de delimitar los bloques que forman parte de un símbolo, para ello UTF-8 se basa en prefijos, existiendo 3 posibilidades:

1. El bloque comienza en 0, que significa que el símbolo esta compuesto por un sólo símbolo. Ej: $(01000010)_2$ o su equivalente en hexadecimal $(42)_{16}$ utiliza un solo bloque de 8 bits y representa el símbolo correspondiente a la letra B.
2. Si un símbolo es representado por más de un bloque, el primer bloque contiene una serie de bits en 1 acorde a la cantidad de bloques totales a utilizar seguido de un 0. Es decir, si el bloque comienza con 11110, significa que se deben decodificar 4 bloques en total, donde se incluye este primero que contiene el prefijo.
3. Los bloques siguientes que forman parte de un grupo mayor comienzan de forma obligatoria con 10, esto no presenta una confusión con el punto 2, dado que un prefijo 10 significaría que el símbolo esta compuesto por un sólo bloque, que no tendría sentido ya que ese caso comienza con un bit en 0.

Analicemos los distintos casos mediante algunos ejemplos de cadenas de bits agrupados en 8:

- 11001000 10111100 es una representación válida de un sólo caracter, dado que el primer bloque comienza con 110 declarando que se requieren 2 bytes y el siguiente comienza con 10 que representa que es un bloque de continuación.
- 11100101 10010111 10100001 00111001 también es una representación válida en este caso de dos símbolos, dado que los tres primeros bloques corresponden al primero ya que comienza con 1110 el primero y es seguido por dos bloques cuyo prefijo es 10, y el último es un caracter de un único bloque dado que comienza en 0.
- 10100010 no es un bloque válido por si solo, ya que comienza por 10 y debería ser la continuación de un conjunto mayor.
- 11001100 por si solo tampoco será un bloque válido ya que falta un bloque extra para poder definir el caracter o símbolo que representa.

3.2. Codificar los identificadores Unicode

Como se menciona en la sección Unicode U+1F9C9 es el identificador asignado al emoji del mate, este símbolo requiere más de un bloque para ser representado, para obtener la representación en UTF-8 de manera manual vamos a realizar una serie de pasos.

1. Convertiremos a binario el identificador, $(1F9C9)_{16} = (0001\ 1111\ 1001\ 1100\ 1001)_2$.
2. Distribuimos los bits en los diferentes bloques, tomando en cuenta que los bloques extra deben comenzar con 10, dejando seis bits disponibles en dichos bloques, y el primer bloque debe comenzar con una serie de 1s igual a la cantidad de bloques totales.

Si reagrupamos de a seis los bits anteriores obtenemos:

00 011111 100111 001001

Entonces, lo que haremos es ir completando bloques con 10 con los bits correspondientes al identificador.

- a) **10001001** será el último bloque que es el prefijo 10 requerido sumado a los últimos 6 bits, es decir parte del hexadecimal C (la parte final 00) y el hexadecimal 9 (1001) completo, C9 no entra por completo dado que requeriría 8 bits.
 - b) **10100111** es el próximo bloque, nuevamente el prefijo de continuación el tercer hexadecimal que es otro 9 (1001) y la otra parte del hexadecimal C (la parte inicial 11).
 - c) **10011111** aca tenemos el prefijo la parte menos significativa del hexadecimal 1 (01) y el hexadecimal F (1111).
 - d) Por último solo quedan los dos bits más significativos del hexadecimal 1 que son 00, este valor será anexado al primer bloque junto con el prefijo que denota la cantidad de bloques totales. Como podemos ver se requieren cuatro bloques en total, por lo tanto el primer bloque será 11110 (los cuatro 1s y el 0) y finalmente se completan los 3 bits restantes con los 0s necesarios el restante de los valores pendientes de codificar, es decir los dos 0 del 1 hexadecimal, resultando en **111100** 0 00, que se muestra separado por espacios a los fines de mejor identificación de lo mencionado en la oración previa.
3. Finalmente agrupamos los 4 bloques obtenidos y volvemos a convertir a hexadecimal para tener su representación en UTF-8:

$(11110000\ 10011111\ 10100111\ 10001001)_2 = (F0\ 9F\ A7\ 89)_{16}$

que es la representación en hexadecimal del emoji del mate en formato UTF-8.

Esto lo podemos comprobar rápidamente en python3 ingresando lo siguiente: `b'\xf0\x9f\xa7\x89'.decode('utf-8')`

Es decir que 1F9C9 es el identificador que es codificado como F09FA789 en UTF-8, puede parecer confuso, pero sencillamente el primero es el número que lo identifica (identificador), pero dado que la computadora debe poder reconstruir el símbolo en base a un formato variable UTF-8 específica como será representado y distribuido en más de un bloque.